

Experiment No 01

August 6, 2026

1 Guided Python Activity: State Space Search

1.1 Objective

Implement a Simple State Space Search (BFS and Greedy BFS) for the 8-Puzzle

1.2 Step 1: Represent the Puzzle

The 8-puzzle is played on a 3x3 grid with 8 numbered tiles and one empty space. We represent the board using a 2D list (a list of lists).

1.2.1 Questions:

1. **Why is 0 used?** It represents the blank (empty) space on the puzzle board.
2. **Why are nested lists used?** To represent a 2D matrix/grid (rows and columns).
3. **How many rows are there?** 3 rows.
4. **How many columns are there?** 3 columns.

```
[1]: # 1. Define Puzzle States
initial_state = [
    [2, 8, 3],
    [1, 6, 4],
    [7, 0, 5]
]

goal_state = [
    [1, 2, 3],
    [8, 0, 4],
    [7, 6, 5]
]

print("--- Initial State ---")
for row in initial_state:
    print(row)
```

```
--- Initial State ---
[2, 8, 3]
[1, 6, 4]
[7, 0, 5]
```

1.3 Step 2: Find the Blank Tile

To move tiles, we need to locate the coordinates (row, col) of the blank tile (0).

1.3.1 Questions:

1. **What does this function return?** A tuple (row, col) representing the exact grid position of 0.
2. **Why do we need the blank position?** Because actions/moves are defined relative to where the blank space is located.

```
[2]: def find_blank(state):  
    for row in range(3):  
        for col in range(3):  
            if state[row][col] == 0:  
                return row, col  
  
r, c = find_blank(initial_state)  
print("Blank Position:", r, c)
```

Blank Position: 2 1

1.4 Step 3: Check Valid Moves

Depending on where the blank tile is located, certain moves might be invalid (e.g., moving UP when at row 0).

1.4.1 Questions:

1. **Why can't the blank move outside the board?** Moving outside index boundaries (0 to 2) causes `IndexError` bugs and breaks the rules of the grid.
2. **How many moves are possible from the centre?** 4 moves (UP, DOWN, LEFT, RIGHT).

```
[3]: def valid_moves(row, col):  
    moves = []  
    if row > 0:  
        moves.append("UP")  
    if row < 2:  
        moves.append("DOWN")  
    if col > 0:  
        moves.append("LEFT")  
    if col < 2:  
        moves.append("RIGHT")  
    return moves  
  
print("Valid Moves from current blank position:", valid_moves(r, c))
```

Valid Moves from current blank position: ['UP', 'LEFT', 'RIGHT']

1.5 Step 4 & 5: Copy State & Move Blank Tile

When transitioning to a new state, we must create a deep copy of the board so we do not modify the original puzzle state.

1.5.1 Questions:

1. **Why do we create a copy instead of changing the original puzzle?** Python lists are mutable and passed by reference. Modifying the state directly destroys the original state in our search tree.
2. **What changed after moving UP?** The blank tile swapped places with the tile immediately above it.

```
[4]: def move_up(state):
    row, col = find_blank(state)
    if row == 0:
        return None # Invalid move

    # Create deep copy
    new_state = [r.copy() for r in state]

    # Swap blank tile with tile above it
    new_state[row][col] = new_state[row-1][col]
    new_state[row-1][col] = 0
    return new_state

def move_down(state):
    row, col = find_blank(state)
    if row == 2:
        return None

    new_state = [r.copy() for r in state]
    new_state[row][col] = new_state[row+1][col]
    new_state[row+1][col] = 0
    return new_state

def move_left(state):
    row, col = find_blank(state)
    if col == 0:
        return None

    new_state = [r.copy() for r in state]
    new_state[row][col] = new_state[row][col-1]
    new_state[row][col-1] = 0
    return new_state

def move_right(state):
    row, col = find_blank(state)
    if col == 2:
        return None
```

```

new_state = [r.copy() for r in state]
new_state[row][col] = new_state[row][col+1]
new_state[row][col+1] = 0
return new_state

```

```

[5]: # Test UP movement
next_state = move_up(initial_state)
print("State after moving UP:")
for row in next_state:
    print(row)

```

State after moving UP:

```

[2, 8, 3]
[1, 0, 4]
[7, 6, 5]

```

1.6 Step 6 & 7: Generate & Display Successor States

A successor function generates all possible valid states reachable from the current state in one move.

1.6.1 Questions:

- How many successor states were generated from initial state? 4 (since blank is in the middle at index (1, 1)).

```

[6]: def generate_successors(state):
    successors = []
    for move_func in [move_up, move_down, move_left, move_right]:
        result = move_func(state)
        if result is not None:
            successors.append(result)
    return successors

successors = generate_successors(initial_state)
print(f"Number of Successors Generated: {len(successors)}\n")

# Display every successor
count = 1
for state in successors:
    print(f"--- Successor {count} ---")
    for row in state:
        print(row)
    count += 1

```

Number of Successors Generated: 3

```

--- Successor 1 ---
[2, 8, 3]
[1, 0, 4]

```

```

[7, 6, 5]
--- Successor 2 ---
[2, 8, 3]
[1, 6, 4]
[0, 7, 5]
--- Successor 3 ---
[2, 8, 3]
[1, 6, 4]
[7, 5, 0]

```

1.7 Extension Activity: Full Breadth-First Search (BFS) Algorithm

Here, we combine all previous steps with a **Queue** (`collections.deque`) to construct a complete **Breadth-First Search** algorithm to find the optimal path to `goal_state`.

```

[7]: from collections import deque

def state_to_tuple(state):
    """Utility to convert 2D list to an immutable tuple so it can be added to
    ↪sets."""
    return tuple(tuple(row) for row in state)

def solve_bfs(start, goal):
    # Store tuples of: (current_state, path_taken)
    queue = deque([(start, [start])])

    # Store visited states as set of tuples for O(1) lookup speed
    visited = {state_to_tuple(start)}

    explored_states = [] # List to record every state as it gets explored

    while queue:
        current_state, path = queue.popleft()
        explored_states.append(current_state) # Record current explored state

        # Check Goal Test
        if current_state == goal:
            return path, explored_states

        # Expand Successor States
        for successor in generate_successors(current_state):
            succ_tuple = state_to_tuple(successor)
            if succ_tuple not in visited:
                visited.add(succ_tuple)
                queue.append((successor, path + [successor]))

    return None, explored_states

```

```

# Run BFS Solver
solution_path, explored_states = solve_bfs(initial_state, goal_state)

# Display Search Summary
print("=====")
print("BFS SEARCH COMPLETED!")
print(f"Total Nodes Explored: {len(explored_states)}")
print(f"Optimal Path Length: {len(solution_path) - 1} moves")
print("=====\\n")

# Display All Explored States
print("ALL EXPLORED STATES (IN ORDER OF EXPANSION):")
print("-----")
for idx, state in enumerate(explored_states, 1):
    print(f"Explored Node {idx}:")
    for row in state:
        print(row)
    print()

# Display Solution Path
print("=====")
print("OPTIMAL SOLUTION PATH:")
print("=====\\n")
for step, state in enumerate(solution_path):
    print(f"--- Step {step} ---")
    for row in state:
        print(row)
    print()

```

```

=====
BFS SEARCH COMPLETED!
Total Nodes Explored: 35
Optimal Path Length: 5 moves
=====

```

```

ALL EXPLORED STATES (IN ORDER OF EXPANSION):
-----

```

Explored Node 1:

[2, 8, 3]

[1, 6, 4]

[7, 0, 5]

Explored Node 2:

[2, 8, 3]

[1, 0, 4]

[7, 6, 5]

Explored Node 3:

[2, 8, 3]
[1, 6, 4]
[0, 7, 5]

Explored Node 4:

[2, 8, 3]
[1, 6, 4]
[7, 5, 0]

Explored Node 5:

[2, 0, 3]
[1, 8, 4]
[7, 6, 5]

Explored Node 6:

[2, 8, 3]
[0, 1, 4]
[7, 6, 5]

Explored Node 7:

[2, 8, 3]
[1, 4, 0]
[7, 6, 5]

Explored Node 8:

[2, 8, 3]
[0, 6, 4]
[1, 7, 5]

Explored Node 9:

[2, 8, 3]
[1, 6, 0]
[7, 5, 4]

Explored Node 10:

[0, 2, 3]
[1, 8, 4]
[7, 6, 5]

Explored Node 11:

[2, 3, 0]
[1, 8, 4]
[7, 6, 5]

Explored Node 12:

[0, 8, 3]
[2, 1, 4]
[7, 6, 5]

Explored Node 13:

[2, 8, 3]

[7, 1, 4]

[0, 6, 5]

Explored Node 14:

[2, 8, 0]

[1, 4, 3]

[7, 6, 5]

Explored Node 15:

[2, 8, 3]

[1, 4, 5]

[7, 6, 0]

Explored Node 16:

[0, 8, 3]

[2, 6, 4]

[1, 7, 5]

Explored Node 17:

[2, 8, 3]

[6, 0, 4]

[1, 7, 5]

Explored Node 18:

[2, 8, 0]

[1, 6, 3]

[7, 5, 4]

Explored Node 19:

[2, 8, 3]

[1, 0, 6]

[7, 5, 4]

Explored Node 20:

[1, 2, 3]

[0, 8, 4]

[7, 6, 5]

Explored Node 21:

[2, 3, 4]

[1, 8, 0]

[7, 6, 5]

Explored Node 22:

[8, 0, 3]

[2, 1, 4]
[7, 6, 5]

Explored Node 23:

[2, 8, 3]
[7, 1, 4]
[6, 0, 5]

Explored Node 24:

[2, 0, 8]
[1, 4, 3]
[7, 6, 5]

Explored Node 25:

[2, 8, 3]
[1, 4, 5]
[7, 0, 6]

Explored Node 26:

[8, 0, 3]
[2, 6, 4]
[1, 7, 5]

Explored Node 27:

[2, 0, 3]
[6, 8, 4]
[1, 7, 5]

Explored Node 28:

[2, 8, 3]
[6, 7, 4]
[1, 0, 5]

Explored Node 29:

[2, 8, 3]
[6, 4, 0]
[1, 7, 5]

Explored Node 30:

[2, 0, 8]
[1, 6, 3]
[7, 5, 4]

Explored Node 31:

[2, 0, 3]
[1, 8, 6]
[7, 5, 4]

Explored Node 32:

[2, 8, 3]

[1, 5, 6]

[7, 0, 4]

Explored Node 33:

[2, 8, 3]

[0, 1, 6]

[7, 5, 4]

Explored Node 34:

[1, 2, 3]

[7, 8, 4]

[0, 6, 5]

Explored Node 35:

[1, 2, 3]

[8, 0, 4]

[7, 6, 5]

=====
OPTIMAL SOLUTION PATH:
=====

--- Step 0 ---

[2, 8, 3]

[1, 6, 4]

[7, 0, 5]

--- Step 1 ---

[2, 8, 3]

[1, 0, 4]

[7, 6, 5]

--- Step 2 ---

[2, 0, 3]

[1, 8, 4]

[7, 6, 5]

--- Step 3 ---

[0, 2, 3]

[1, 8, 4]

[7, 6, 5]

--- Step 4 ---

[1, 2, 3]

[0, 8, 4]

[7, 6, 5]

```
--- Step 5 ---  
[1, 2, 3]  
[8, 0, 4]  
[7, 6, 5]
```